

DOKUMENTASI TEKNIS
SIMPels



Sistem Informasi Manajemen Pemeliharaan
Instrumen

Informasi Pengembang dan Hak Cipta

Identitas Aplikasi

Nama Aplikasi	SIMPels (Sistem Informasi Manajemen Pemeliharaan Instrumen)
Versi	1.0 (Stable Build)
Tanggal Rilis	Desember 2025
Basis Pengembangan	Python (Tkinter), SQLite, ReportLab
Pemilik Sistem	BMKG - Stasiun Geofisika Sleman

Hak Cipta (Copyright)

© 2025 Stasiun Geofisika Sleman – Badan Meteorologi, Klimatologi, dan Geofisika.

Seluruh hak cipta dilindungi undang-undang.

Dokumen teknis dan perangkat lunak ini dikembangkan sebagai bagian dari kegiatan Kerja Praktik Mahasiswa.

Kepemilikan penuh atas:

- kode sumber (*source code*),
- basis data,
- aset visual,
- dan seluruh modul aplikasi

sepenuhnya berada di bawah wewenang **Stasiun Geofisika Sleman**.

Dilarang memperbanyak, mendistribusikan, atau memodifikasi sebagian maupun seluruh isi dokumen dan aplikasi untuk kepentingan komersial tanpa izin tertulis dari Kepala Stasiun Geofisika Sleman.

Penggunaan aplikasi terbatas untuk operasional internal BMKG.

Kredit Pengembang (Credits)

Pengembang Utama (Lead Developer)

Andrea Marie Baikole
Mahasiswa Program Studi Informatika
Universitas Kristen Duta Wacana (UKDW)

Supervisor Lapangan

Wiji Karyanti, S.T
Stasiun Geofisika Sleman – BMKG

Dosen Pembimbing

Restyandito, S.Kom, MSIS., Ph.D
Fakultas Teknologi Informasi, UKDW

Daftar Isi Dokumentasi Teknis

1. Overview Aplikasi.....	6
Deskripsi Singkat Aplikasi.....	6
Tujuan Pengembangan.....	6
Arsitektur Umum Sistem.....	6
2. Arsitektur Program.....	7
Diagram Alur Data & Sistem.....	7
Struktur Direktori.....	9
3. Modul Utama Aplikasi.....	10
a. Modul Database (services/.....)	10
b. Modul Generator PDF (services/generate_*.py).....	10
c. Modul Antarmuka Pengguna (ui/.....)	11
4. Dokumentasi Skema Database Pemeliharaan.....	12
ERD Diagram.....	12
Struktur Tabel.....	13
Master Data.....	13
Laporan Utama.....	16
Detail Laporan Seismograph.....	18
Detail Laporan Preventif.....	21
Detail Laporan Umum (WRS NG).....	23
Tabel Pendukung.....	25
Catatan Konfigurasi Database.....	27
Indeks Database.....	27
Relasi Antar Tabel.....	27
Relasi One-to-Many.....	27
Relasi Many-to-Many.....	27
Cascade Delete.....	27
5. Build, Run, & Deployment Guide.....	28
Cara Running di Local (Tahap Development).....	28
Cara Build Aplikasi.....	28
6. Panduan Manajemen Database Manual.....	29
a. Instalasi DB Browser for SQLite.....	29
b. Membuka Database Aplikasi.....	29
c. Menambah Data Baru (Insert).....	29
d. Memperbarui Data (Update).....	30
e. Menghapus Data (Delete) & Peringatan Cascade.....	30
7. Panduan Pengembangan Lanjutan.....	32
Skenario 1: Menambah Kolom Baru di Database.....	32

Skenario 2: Menambahkan Modul Halaman Baru.....	33
Skenario 3: Mengganti Kop Surat.....	33
Skenario 4: Menambah Opsi Label pada Dropdown (Statis).....	34
Skenario 5: Implementasi Input dengan Auto-complete.....	36

1. Overview Aplikasi

Deskripsi Singkat Aplikasi

SIMPels (Sistem Informasi Manajemen Pemeliharaan Instrumen) adalah aplikasi perangkat lunak berbasis desktop yang dikembangkan menggunakan bahasa pemrograman Python dan kerangka kerja (framework) GUI Tkinter. Aplikasi ini dirancang untuk memfasilitasi digitalisasi, pencatatan, dan pelaporan kegiatan pemeliharaan instrumen geofisika (Seismograph, Accelerograph, Intensitymeter, dan WRS NG).

Tujuan Pengembangan

1. **Standarisasi Pelaporan:** Menyeragamkan format laporan pemeliharaan sesuai standar operasional BMKG.
2. **Efisiensi Operasional:** Mempercepat proses pembuatan laporan melalui fitur *autocomplete* dan *template* otomatis.
3. **Integritas Data:** Menyimpan riwayat pemeliharaan dan bukti dukung (foto) dalam basis data terpusat yang aman.
4. **Otomasi Dokumen:** Menghasilkan dokumen fisik (PDF) siap cetak secara otomatis dari data yang diinput.

Arsitektur Umum Sistem

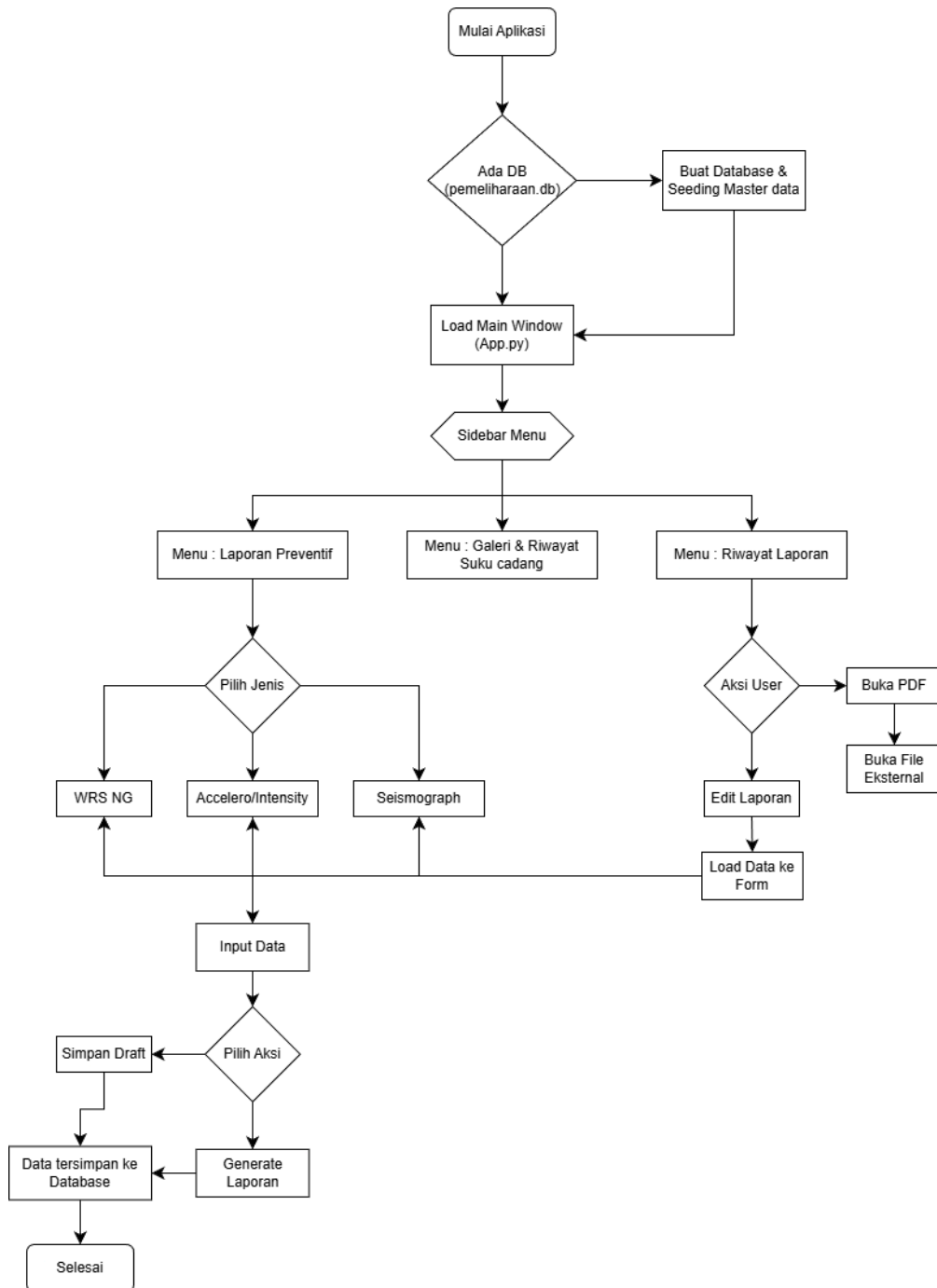
Sistem ini menerapkan pola desain **Modular MVC (Model-View-Controller)** yang dimodifikasi untuk lingkungan desktop *stateful*:

- **Model (Service Layer):** Menangani logika bisnis, akses basis data (SQLite), dan pembuatan dokumen (ReportLab).
- **View (UI Layer):** Menangani tata letak visual dan interaksi pengguna menggunakan `ttk` widgets.
- **Controller:** Bertindak sebagai perantara yang memproses input dari View, memvalidasi data, dan meneruskannya ke Model.

2. Arsitektur Program

Diagram Alur Data & Sistem

Diagram berikut menggambarkan siklus hidup data (*Data Lifecycle*) mulai dari interaksi pengguna hingga menjadi dokumen fisik dan arsip database:



Penjelasan Teknis Alur Data:

Proses pengolahan data dalam SIMPels mengikuti pola arsitektur MVC (Model-View-Controller) dengan rincian sebagai berikut:

1. Fase Input & Validasi (View Layer):

- Pengguna berinteraksi melalui antarmuka (*View*) seperti **HalamanForm** atau **HalamanSeismograph**.
- Data ditangkap dari berbagai *widget* (Entry, Combobox, RichTextEditor).
- **Validasi Awal:** Sebelum data dikirim, sistem melakukan pengecekan integritas dasar di sisi UI (misal: memastikan kolom bertanda bintang merah * tidak kosong).

2. Fase Orkestrasi (Controller Layer):

- **Controller** bertindak sebagai jembatan logika. Ia mengumpulkan data mentah dari UI menggunakan metode seperti `_collect_data()`.
- Controller melakukan sanitasi data (misal: mengubah format tanggal UI `dd-mm-yyyy` menjadi format database `yyyy-mm-dd`).
- Logika bisnis diterapkan di sini, misalnya: *"Jika Kode Stasiun belum ada di master data, buat baru secara otomatis sebelum menyimpan laporan."*

3. Fase Persistensi (Service/Model Layer):

- Controller mengirimkan data bersih ke `db_service`.
- **ACID Transaction:** `db_service` membuka blok transaksi atomik (`@db_transaction`).
- **Splitting:** Data dipecah dan didistribusikan ke tabel yang relevan:
 - Metadata utama masuk ke tabel `laporan`.
 - Data teknis masuk ke tabel detail (misal: `seismograph_detail`).
 - Data list (petugas, foto, alat) masuk ke tabel relasi *one-to-many* (misal: `laporan_petugas`).
- Jika terjadi error di tengah jalan, seluruh operasi dibatalkan (*Rollback*) untuk mencegah data korup.

4. Fase Generasi Dokumen (Reporting Engine):

- Jika mode "Generate" dipilih, data diteruskan ke modul **GenerateLaporan**.
- **Rendering:** Library **ReportLab** menggambar ulang data teks dan foto ke atas kanvas digital (PDF).
- **Merging:** Jika pengguna melampirkan file eksternal (Surat Tugas), library **PyPDF** akan menggabungkannya dengan halaman laporan utama menjadi satu file utuh.

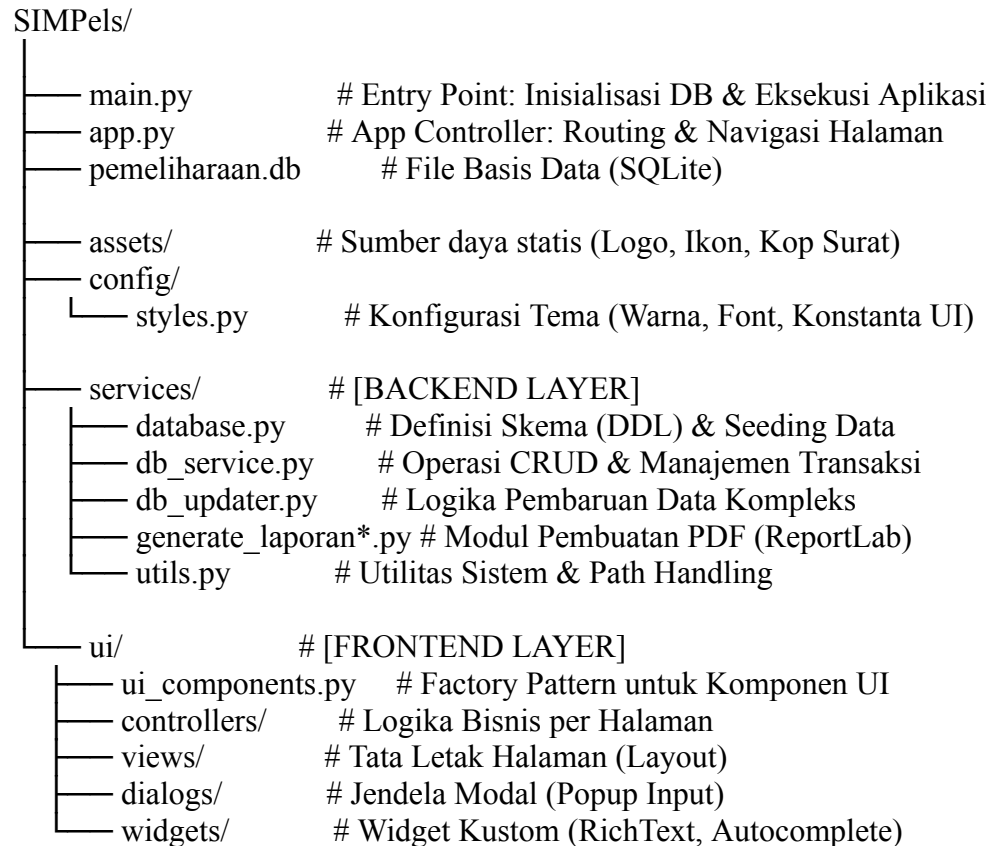
5. Fase Sinkronisasi (Broadcast Event):

- Setelah data tersimpan sukses, **App** memicu sinyal *broadcast*.

- Seluruh modul lain (History, Galeri, Suku Cadang) yang sedang aktif akan menerima sinyal ini dan melakukan *refresh* data secara otomatis, sehingga data yang ditampilkan selalu *real-time*.

Struktur Direktori

Struktur kode sumber diorganisir untuk memisahkan tanggung jawab



3. Modul Utama Aplikasi

Berikut adalah rincian komponen penyusun aplikasi SIMPels beserta fungsinya:

a. Modul Database (services/)

Apa fungsinya? Ini adalah "**Gudang Penyimpanan**" aplikasi. Modul ini bertugas memastikan semua data yang diinput (laporan, foto, data teknisi) tersimpan aman ke dalam file database (**.db**). Tanpa modul ini, data akan hilang begitu aplikasi ditutup.

- **db_service.py**: Bertindak sebagai **pintu gerbang** keluar-masuk data. File ini menggunakan pola *Decorator* (**@db_transaction**) untuk menjaga keamanan koneksi. Ia memastikan pintu database dibuka saat mau simpan data, dan *pasti* ditutup lagi setelah selesai (atau jika terjadi error), sehingga database tidak "macet" atau rusak (corrupt).
- **db_updater.py**: File ini adalah **spesialis update data kompleks**. Ia menangani data yang punya hubungan "satu induk – banyak anak" (misal: 1 Laporan punya 5 Foto).
 - *Cara kerjanya*: Saat Anda mengedit laporan, modul ini tidak memilah satu-satu foto mana yang berubah. Sebaliknya, ia menggunakan metode "*Wipe & Rewrite*": hapus semua foto lama terkait laporan tersebut, lalu simpan ulang daftar foto yang baru. Ini menjamin data selalu bersih, rapi, dan konsisten tanpa ada data "hantu" yang tertinggal.

b. Modul Generator PDF (services/generate_*.py)

Apa fungsinya? Ini adalah "**Mesin Percetakan Otomatis**". Tugasnya mengambil data mentah dari formulir yang Anda isi, lalu menyusunnya menjadi dokumen PDF resmi yang rapi dan siap cetak sesuai standar laporan BMKG.

Menggunakan *library* **ReportLab**, modul ini memiliki kemampuan:

- **Tata Letak Cerdas**: Bisa mengatur tabel panjang agar otomatis pindah ke halaman baru (*pagination*) jika halaman pertama penuh, tanpa memotong teks sembarangan.
- **Gambar Dinamis**: Menampilkan foto dokumentasi dengan ukuran yang proporsional di dalam tabel.
- **Rich Text**: Mendukung format teks seperti **tebal**, *miring*, atau daftar poin (bullet points) pada kolom rekomendasi/catatan.

- **Penggabungan Dokumen (Integration):** Menggunakan pustaka `pypdf` untuk "menjahit" dokumen tambahan (seperti lampiran Surat Tugas) menjadi satu kesatuan file PDF dengan laporan utama.

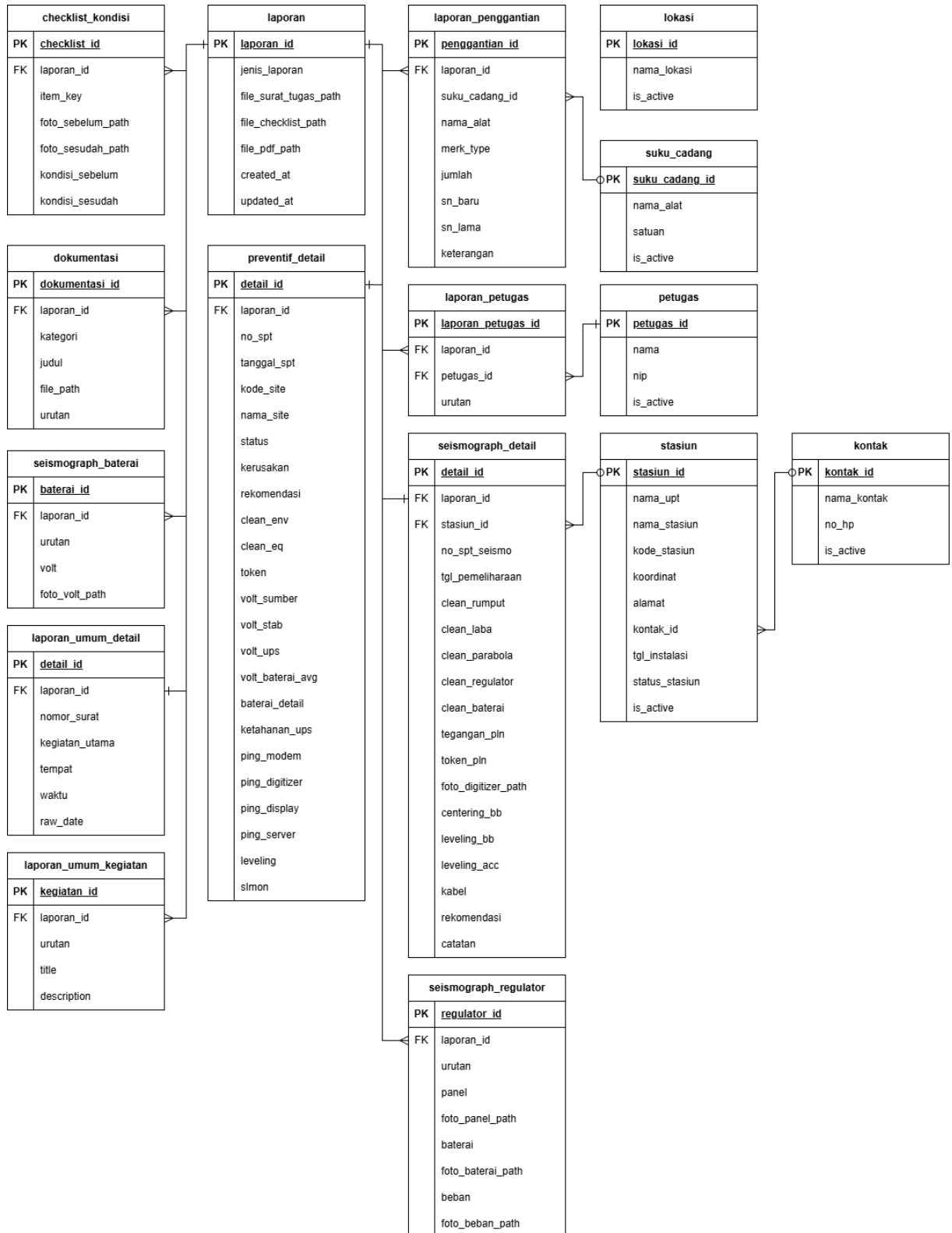
c. Modul Antarmuka Pengguna (ui/)

Apa fungsinya? Ini adalah "**Wajah dan Otak**" interaksi aplikasi. Modul ini mengatur apa yang Anda lihat di layar (View) dan apa yang terjadi saat Anda menekan tombol (Controller).

- **Controller (Otak):** Mengatur logika bisnis dan validasi.
 - *Contoh kecerdasannya:* Pada `SeismographController`, saat Anda mengetik Kode Stasiun baru yang belum terdaftar, controller tidak akan menolak error. Sebaliknya, ia akan **otomatis menyimpan** stasiun baru tersebut ke data master, sehingga Anda tidak perlu input dua kali.
- **Asynchronous Gallery (Kinerja):** Menangani tampilan Galeri Foto.
 - *Masalah:* Memuat ratusan foto sekaligus biasanya membuat aplikasi macet (*freeze/not responding*).
 - *Solusi:* `GalleryController` menggunakan teknik **Multithreading** (kerja paralel). Ia memuat foto di "belakang layar" (background) satu per satu. Hasilnya, aplikasi tetap licin dan responsif bisa digulir (*scroll*) meskipun gambar sedang proses dimuat.

4. Dokumentasi Skema Database Pemeliharaan

ERD Diagram



Struktur Tabel

Master Data

1. Petugas

Menyimpan data petugas pemeliharaan.

Kolom	Tipe	Keterangan
petugas_id	INTEGER (PK)	Primary Key, Auto Increment
nama	TEXT	Nama petugas (NOT NULL)
nip	TEXT	Nomor Induk Pegawai
is_active	INTEGER	Status aktif, default: 1

Constraint : UNIQUE (nama, nip)

2. kontak

Menyimpan data kontak atau penanggung jawab stasiun

Kolom	Tipe	Keterangan
kontak_id	INTEGER (PK)	Primary Key, Auto Increment
nama_kontak	TEXT	Nama orang yang dapat dihubungi di stasiun tersebut (NOT NULL)
no_hp	TEXT	Nomor HP
is_active	INTEGER	Status aktif, default: 1

Constraint : UNIQUE (nama_kontak, no_hp)

3. stasiun

Menyimpan informasi stasiun seismograph, accelerograph dan intensitymeter

Kolom	Tipe	Keterangan
stasiun_id	INTEGER (PK)	Primary Key, Auto Increment
nama_upt	TEXT	Nama unit pelaksana teknis

nama_stasiun	TEXT	Nama stasiun (NOT NULL)
kode_stasiun	TEXT (UNIQUE)	Kode unik stasiun (NOT NULL, UNIQUE)
koordinat	TEXT	Koordinat GPS (lat, long)
alamat	TEXT	Alamat lengkap stasiun
kontak_id	INTEGER (FK)	Foreign Key ke tabel kontak
tgl_instalasi	TEXT	Tanggal instalasi stasiun (YYYY-MM-DD)
status_stasiun	TEXT	Status operasional stasiun
is_active	INTEGER	Status aktif, default: 1

Foreign Key : kontak_id REFERENCES kontak(kontak_id) ON DELETE SET NULL

Index : idx_stasiun_kode pada kolom kode_stasiun

4. lokasi

Data referensi lokasi kegiatan WRS NG.

Kolom	Tipe	Keterangan
lokasi_id	INTEGER (PK)	Primary Key, Auto Increment
nama_lokasi	TEXT	Nama lokasi WRS NG (NOT NULL, UNIQUE)
is_active	INTEGER	Status aktif, default: 1

5. suku_cadang

Menyimpan daftar suku cadang yang diubah dalam pemeliharaan preventif maupun mandiri.

Kolom	Tipe	Keterangan
suku_cadang_id	INTEGER (PK)	Primary Key, Auto Increment

nama_alat	TEXT (UNIQUE)	Nama alat/suku cadang yang diganti (NOT NULL, UNIQUE)
satuan	TEXT	Satuan (misal: pcs, unit)
is_active	INTEGER	Status aktif, default: 1

Laporan Utama

1. laporan

Tabel induk untuk mendaftarkan semua jenis laporan yang dibuat.

Kolom	Tipe	Keterangan
laporan_id	INTEGER (PK)	Primary Key, Auto Increment
jenis_laporan	TEXT	Jenis laporan (seismograph, accelograph, intensitymeter, umum), NOT NULL
file_surat_tugas_path	TEXT	Path file surat tugas
file_checklist_path	TEXT	Path file checklist kegiatan
file_pdf_path	TEXT	Path laporan PDF yang digenerate
created_at	TEXT	Waktu pembuatan laporan
updated_at	TEXT	Waktu terakhir laporan diperbarui

Index :

- idx_laporan_jenis pada kolom jenis_laporan
- Idx_laporan_updated pada kolom updated_at

2. laporan_petugas

Tabel relasi yang menghubungkan laporan dan petugas

Kolom	Tipe	Keterangan
laporan_petugas_id	INTEGER (PK)	Primary Key, Auto Increment
laporan_id	INTEGER (FK)	Foreign Key ke tabel laporan (NOT NULL)
petugas_id	INTEGER (FK)	Foreign Key ke tabel petugas (NOT NULL)
urutan	INTEGER	Urutan petugas dalam

		tim
--	--	-----

Foreign Keys :

- laporan_id REFERENCES laporan(laporan_id) ON DELETE CASCADE
- petugas_id REFERENCES petugas(petugas_id) ON DELETE RESTRICT

Constraint : UNIQUE(laporan_id, petugas_id)

3. laporan_penggantian

Menyimpan data penggantian suku cadang dalam laporan.

Kolom	Tipe	Keterangan
penggantian_id	INTEGER	Primary Key, Auto Increment
laporan_id	INTEGER	Foreign Key ke tabel laporan (NOT NULL)
suku_cadang_id	INTEGER	Foreign Key ke tabel suku_cadang
nama_alat	TEXT	Nama alat yang diganti
merk_type	TEXT	Merk atau tipe alat
jumlah	INTEGER	Jumlah yang diganti (default: 1, NOT NULL)
sn_baru	TEXT	Serial Number baru
sn_lama	TEXT	Serial Number lama
keterangan	TEXT	Keterangan tambahan

Foreign Keys :

- laporan_id REFERENCES laporan(laporan_id) ON DELETE CASCADE
- suku_cadang_id REFERENCES suku_cadang(suku_cadang_id) ON DELETE SET NULL

Detail Laporan Seismograph

1. seismograph_detail

Isi data keseluruhan dari laporan preventif maintenance seismograph.

Kolom	Tipe	Keterangan
detail_id	INTEGER (PK)	Primary Key, Auto Increment
laporan_id	INTEGER (FK, UNIQUE)	Foreign Key ke tabel laporan (NOT NULL, UNIQUE)
stasiun_id	INTEGER (FK)	Foreign Key ke tabel stasiun
no_spt_seismo	TEXT	Nomor surat perintah tugas
tgl_pemeliharaan	TEXT	Tanggal Pemeliharaan (YYYY-MM-DD)
clean_rumput	TEXT	Status pembersihan rumput
clean_laba	TEXT	Status pembersihan sarang laba-laba
clean_parabola	TEXT	Status pembersihan parabola
clean_regulator	TEXT	Status pembersihan regulator
clean_baterai	TEXT	Status pembersihan baterai
tegangan_pln	TEXT	Besaran tegangan PLN (Volt)
token_pln	TEXT	Sisa Token PLN
foto_digitizer_path	TEXT	Path foto digitizer
centering_bb	TEXT	Status centering broadband
leveling_bb	TEXT	Status leveling broadband

leveling_acc	TEXT	Status leveling accelerometer
kabel	TEXT	Kondisi kabel
rekomendasi	TEXT	Rekomendasi perbaikan
catatan	TEXT	Catatan tambahan

Foreign Keys :

- laporan_id REFERENCES laporan(laporan_id) ON DELETE CASCADE
- stasiun_id REFERENCES stasiun(stasiun_id) ON DELETE SET NULL

2. seismograph_regulator

Menyimpan data hasil pengukuran regulator dalam kegiatan preventif maintenance seismograph

Kolom	Tipe	Keterangan
regulator_id	INTEGER (PK)	Primary Key, Auto Increment
laporan_id	INTEGER (FK)	Foreign Key ke tabel laporan (NOT NULL)
urutan	INTEGER	Urutan panel regulator
panel	TEXT	Nilai tegangan panel
foto_panel_path	TEXT	Path foto panel
baterai	TEXT	Nilai tegangan baterai
foto_baterai_path	TEXT	Path foto baterai
beban	TEXT	Nilai beban (Ampere)
foto_beban_path	TEXT	Path foto beban

Foreign Key : laporan_id REFERENCES laporan(laporan_id) ON DELETE CASCADE

3. seismograph_baterai

Menyimpan data hasil pengukuran baterai dalam kegiatan preventif maintenance seismograph

Kolom	Tipe	Keterangan
-------	------	------------

baterai_id	INTEGER (PK)	Primary Key, Auto Increment
laporan_id	INTEGER (FK)	Foreign Key ke tabel laporan (NOT NULL)
urutan	INTEGER	Urutan baterai
volt	TEXT	Tegangan baterai (volt)
foto_volt_path	TEXT	Path foto voltmeter baterai

Foreign Key : laporan_id REFERENCES laporan(laporan_id) ON DELETE CASCADE

Detail Laporan Preventif

1. preventif_detail

Isi data keseluruhan dari laporan preventif maintenance accelerometer/intensitymeter.

Kolom	Tipe	Keterangan
detail_id	INTEGER (PK)	Primary Key, Auto Increment
laporan_id	INTEGER (FK, UNIQUE)	Foreign Key ke tabel laporan (NOT NULL, UNIQUE)
no_spt	TEXT	Nomor surat perintah tugas
tanggal_spt	TEXT	Tanggal SPT (YYYY-MM-DD)
kode_site	TEXT	Kode lokasi pemeliharaan
nama_site	TEXT	Nama tempat pemeliharaan
status	TEXT	Status tempat pemeliharaan (ON/OFF)
kerusakan	TEXT	Deskripsi kerusakan
rekomendasi	TEXT	Rekomendasi tindakan
clean_env	TEXT	Status kebersihan lingkungan
clean_eq	TEXT	Status kebersihan peralatan
token	TEXT	Jumlah token tersisa/terisi
volt_sumber	TEXT	Nilai tegangan sumber
volt_stab	TEXT	Nilai tegangan stabilizer
volt_ups	TEXT	Nilai tegangan UPS
volt_baterai_avg	TEXT	Rata-rata tegangan

		baterai
baterai_detail	TEXT	Nilai tegangan pada beberapa baterai
ketahanan_ups	TEXT	Status ketahanan UPS
ping_modem	TEXT	Status ping modem
ping_digitizer	TEXT	Status ping digitizer
ping_display	TEXT	Status ping display
ping_server	TEXT	Status ping server
leveling	TEXT	Status leveling sensor
slmon	TEXT	Status SLMON

Foreign Key : laporan_id REFERENCES laporan(laporan_id) ON DELETE CASCADE

Detail Laporan Umum (WRS NG)

1. laporan_umum_detail

Isi data keseluruhan dari laporan pemeliharaan mandiri WRS NG.

Kolom	Tipe	Keterangan
detail_id	INTEGER (PK)	Primary Key, Auto Increment
laporan_id	INTEGER (FK, UNIQUE)	Foreign Key ke tabel laporan (NOT NULL, UNIQUE)
nomor_surat	TEXT	Nomor surat perintah tugas
kegiatan_utama	TEXT	Deskripsi kegiatan utama
tempat	TEXT	Lokasi kegiatan
waktu	TEXT	Tanggal pelaksanaan pemeliharaan mandiri (formatted)
raw_date	TEXT	Tanggal mentah sebelum diconvert ke waktu

Foreign Key : laporan_id REFERENCES laporan(laporan_id) ON DELETE CASCADE

2. laporan_umum_kegiatan

Daftar kegiatan dan detailnya dalam pemeliharaan mandiri WRS NG.

Kolom	Tipe	Keterangan
kegiatan_id	INTEGER (PK)	Primary Key, Auto Increment
laporan_id	INTEGER (FK)	Foreign Key ke tabel laporan (NOT NULL)
urutan	INTEGER	Urutan kegiatan
title	TEXT	Judul kegiatan (NOT NULL)
description	TEXT	Deskripsi/detail kegiatan tiap judul

Foreign Key : laporan_id REFERENCES laporan(laporan_id) ON DELETE
CASCADE

Tabel Pendukung

1. checklist_kondisi

Daftar checklist kondisi sebelum dan sesudah kegiatan pemeliharaan.

Kolom	Tipe	Keterangan
checklist_id	INTEGER (PK)	Primary Key, Auto Increment
laporan_id	INTEGER (FK)	Foreign Key ke tabel laporan (NOT NULL)
item_key	TEXT	Key item checklist (NOT NULL)
foto_sebelum_path	TEXT	Path foto kondisi sebelum kegiatan
foto_sesudah_path	TEXT	Path foto kondisi setelah kegiatan
kondisi_sebelum	TEXT	Deskripsi kondisi sebelum kegiatan
kondisi_sesudah	TEXT	Deskripsi kondisi setelah kegiatan

Foreign Key : laporan_id REFERENCES laporan(laporan_id) ON DELETE CASCADE

2. dokumentasi

Database penyimpanan path foto kegiatan dan peralatan pada laporan WRS NG

Kolom	Tipe	Keterangan
dokumentasi_id	INTEGER (PK)	Primary Key, Auto Increment
laporan_id	INTEGER (FK)	Foreign Key ke tabel laporan (NOT NULL)
kategori	TEXT	Kategori : peralatan atau kegiatan (NOT NULL)
judul	TEXT	Judul/caption foto
file_path	TEXT	Path file foto (NOT NULL)

urutan	INTEGER	Urutan tampilan foto
--------	---------	----------------------

Foreign Key : laporan_id REFERENCES laporan(laporan_id) ON DELETE
CASCADE

Catatan Konfigurasi Database

- **Database Engine:** SQLite3
- **Foreign Keys:** DIAKTIFKAN (PRAGMA foreign_keys = ON)
- **Row Factory:** sqlite3.Row (untuk akses data seperti dictionary)
- **Timezone:** localtime (waktu lokal sistem)
- **File Database:** pemeliharaan.db

Indeks Database

Index database digunakan untuk mempercepat pencarian sebuah data

Nama Indeks	Tabel	Kolom	Tujuan
idx_laporan_jenis	laporan	jenis_laporan	Optimasi query filter berdasarkan jenis laporan
idx_laporan_updated	laporan	updated_at	Optimasi query sorting berdasarkan waktu update
idx_stasiun_kode	stasiun	kode_stasiun	Optimasi pencarian stasiun berdasarkan kode

Relasi Antar Tabel

Relasi One-to-Many

- **kontak** → **stasiun** (satu kontak bisa untuk banyak stasiun)
- **laporan** → **laporan_petugas** (satu laporan bisa melibatkan banyak petugas)
- **laporan** → **laporan_penggantian** (satu laporan bisa memiliki banyak penggantian)
- **laporan** → berbagai tabel detail (seismograph_detail, preventif_detail, dll)

Relasi Many-to-Many

- **laporan** ↔ **petugas** (melalui tabel **laporan_petugas**)

Cascade Delete

Ketika record di tabel **laporan** dihapus, semua data terkait di tabel berikut akan ikut terhapus:

- laporan_petugas
- laporan_penggantian
- seismograph_detail, seismograph_regulator, seismograph_baterai
- preventif_detail
- laporan_umum_detail, laporan_umum_kegiatan
- checklist_kondisi
- dokumentasi

5. Build, Run, & Deployment Guide

Cara Running di Local (Tahap Development)

1. Pastikan Python 3.12.3 terinstall
2. Buat virtual environment dan aktifkan venvnya.

```
python -m venv venv
source venv/bin/activate # (atau . venv/Scripts/activate
di Windows)
```

3. Install seluruh dependensi eksternal yang dibutuhkan

```
pip install -r requirements.txt
```

4. Jalankan aplikasi dari root directory

```
python main.py
```

5. Aplikasi akan berjalan dan file **pemeliharaan.db** akan dibuat secara otomatis.

Cara Build Aplikasi

Aplikasi ini dirancang untuk di-*build* menggunakan **PyInstaller** (v6.16.0 atau yang kompatibel).

1. Instal PyInstaller dan dependensinya (jika belum):

```
pip install pyinstaller
```

2. Jalankan perintah build dari root directory

```
pyinstaller --onefile --windowed --name SIMPels \
--icon assets/logo_aplikasi.png \
--add-data "assets;assets" main.py
```

Keterangan :

- **--windowed**: Menghilangkan jendela konsol.
- **--onefile**: (Opsional) Mengemas semuanya menjadi satu file **.exe** tunggal.
- **--add-data "assets;assets"**: **Krusial**. Ini memastikan folder **assets** (berisi kop surat dan logo) di-*bundle* bersama aplikasi.
- **--icon**: (Opsional) Menambahkan ikon aplikasi.

3. Hasil *build* akan tersedia di folder **dist/SIMPEL.exe**.

6. Panduan Manajemen Database Manual

Bagian ini menjelaskan cara mengakses, mengubah, dan memelihara data di dalam file **pemeliharaan.db** secara manual menggunakan perangkat lunak pihak ketiga. Hal ini berguna untuk *debugging*, koreksi data massal, atau reset password (jika ada) tanpa melalui antarmuka aplikasi.

a. Instalasi DB Browser for SQLite

Kami merekomendasikan penggunaan **DB Browser for SQLite** karena ringan, *open-source*, dan kompatibel dengan *engine* SQLite3 yang digunakan aplikasi.

1. Unduh Aplikasi:

- Kunjungi situs resmi: <https://sqlitebrowser.org/dl/>
- Pilih installer yang sesuai dengan sistem operasi Anda (Windows 64-bit / 32-bit).
- Selain itu anda juga bisa menjalankan installer yang sudah saya siapkan pada nama folder *database* dengan nama file [DB.Browser.for.SQLite-v3.13.1-win64](#)

2. Instalasi:

- Jalankan file installer **.msi** atau **.exe** yang telah diunduh.
- Ikuti instruksi instalasi (klik *Next* hingga *Finish*).
- Pilih opsi untuk membuat *shortcut* di Desktop untuk memudahkan akses.

b. Membuka Database Aplikasi

- 1) Pastikan aplikasi **SIMPels** sedang **tidak berjalan/ditutup**. Hal ini penting untuk mencegah konflik penguncian file (*file locking*).
- 2) Buka aplikasi **DB Browser for SQLite**.
- 3) Klik tombol **"Open Database"** di pojok kiri atas.
- 4) Navigasikan ke folder proyek aplikasi SIMPels.
- 5) Pilih file database bernama **pemeliharaan.db** yang terletak di *root directory*.
- 6) Klik **Open**. Struktur tabel akan muncul di tab "Database Structure".

c. Menambah Data Baru (Insert)

Skenario ini digunakan jika Anda ingin menambahkan *Master Data* (seperti Petugas atau Stasiun) tanpa melalui aplikasi.

1. Pilih tab **"Browse Data"**.
2. Pilih tabel yang ingin ditambah datanya pada *dropdown* "Table" (contoh: pilih tabel **petugas**).

3. Klik tombol **"New Record"** (ikon kertas dengan tanda plus). Baris baru akan muncul.
4. Isi kolom-kolom yang tersedia:
 - **petugas_id**: Biarkan kosong atau (**AutoIncrement**), database akan mengisinya otomatis.
 - **nama**: Ketik nama petugas.
 - **nip**: Ketik NIP petugas.
 - **is_active**: Isi dengan angka **1** (aktif).
5. Klik tombol **"Write Changes"** di bagian atas untuk menyimpan perubahan secara permanen ke file **.db**.

d. Memperbarui Data (Update)

Skenario ini digunakan untuk mengedit kesalahan ketik (typo) atau mengubah status data.

1. Pilih tab **"Browse Data"**.
2. Pilih tabel target (misal: **stasiun**).
3. Gunakan kolom **Filter** di bawah nama kolom untuk mencari data spesifik (misal: cari **Sleman** di kolom **nama_stasiun**).
4. Klik dua kali (*double click*) pada sel yang ingin diubah.
5. Panel editor akan muncul di sebelah kanan. Ubah nilainya sesuai kebutuhan.
 - *Contoh*: Mengubah **status_stasiun** dari "Rusak" menjadi "Baik".
6. Klik tombol **Apply** pada panel editor.
7. Terakhir, jangan lupa klik **"Write Changes"** agar perubahan tersimpan.

e. Menghapus Data (Delete) & Peringatan Cascade

1. Pilih tab **"Browse Data"**.
2. Pilih tabel target.
3. Klik pada nomor baris (di paling kiri) untuk menyeleksi seluruh baris data.
4. Klik tombol **"Delete Record"** (ikon tempat sampah).
5. Baris akan berwarna merah (tanda akan dihapus).
6. Klik **"Write Changes"** untuk mengeksekusi penghapusan.

 **PERINGATAN KRUSIAL:** Sistem ini mengaktifkan **PRAGMA foreign_keys = ON** dan aturan **ON DELETE CASCADE**.

Artinya: Jika Anda menghapus **1 baris** di tabel **laporan**, maka **seluruh data** di tabel berikut yang terhubung dengan laporan ID tersebut akan **ikut terhapus otomatis**: * Detail Laporan (**seismograph_detail**, **preventif_detail**, dll). * Daftar Petugas di laporan tersebut (**laporan_petugas**). * Daftar Suku

Cadang yang diganti ([laporan_penggantian](#)). * Seluruh Foto Dokumentasi terkait ([dokumentasi](#)).

Pastikan Anda benar-benar yakin sebelum menghapus data pada tabel induk ([laporan](#)).

7. Panduan Pengembangan Lanjutan

Bagian ini berisi skenario teknis untuk memandu pengembang selanjutnya dalam melakukan pemeliharaan atau penambahan fitur pada sistem.

Skenario 1: Menambah Kolom Baru di Database

Kasus: Menambahkan kolom "Suhu Ruangan" pada laporan Seismograph.

1. Update Skema Database (**services/database.py**):

- Cari variabel **SCHEMA**.
- Pada definisi tabel **CREATE TABLE IF NOT EXISTS seismograph_detail**, tambahkan: **suhu_ruangan TEXT**,
- *Catatan:* Untuk database yang sudah berjalan (existing), Anda harus melakukan **ALTER TABLE** manual menggunakan *DB Browser for SQLite* atau membuat skrip migrasi, karena inisialisasi ulang tidak akan mengubah struktur tabel yang sudah ada.

2. Update Service Layer (**services/db_service.py** & **db_updater.py**):

- Pada fungsi **save_laporan_seismo** (**db_service.py**), tambahkan parameter **suhu_ruangan** ke dalam query **INSERT**.
- Pada fungsi **update_laporan_seismo** (**db_updater.py**), tambahkan **suhu_ruangan=?** ke dalam query **UPDATE**.

3. Update Controller (**ui/controllers/seismograph_controller.py**):

- Pada metode **_collect_all_data_from_ui**, tambahkan logika pengambilan data:

```
"suhu_ruangan": self.view.entry_suhu.get(),
```
- Pada metode **load_data_for_edit**, tambahkan logika pengisian data ke UI:

```
self.view.entry_suhu.insert(0,  
data.get('suhu_ruangan', ''))
```

4. Update View (**ui/views/halaman_seismograph.py**):

- Tambahkan widget input baru menggunakan komponen UI standar:

```
self.entry_suhu =  
ui_components.create_measurement_row(frame, "Suhu  
Ruangan", "°C", row_idx)
```

5. Update PDF Generator (**services/generate_laporan_seismo.py**):

- Tambahkan baris baru pada tabel laporan untuk menampilkan data suhu.

Skenario 2: Menambahkan Modul Halaman Baru

Kasus: Menambahkan menu "Laporan Magnet Bumi".

1. Buat View Baru:

- Duplikasi file `ui/views/halaman_form.py` dan beri nama `ui/views/halaman_magnet.py`.
- Ubah nama kelas menjadi `HalamanMagnet` dan sesuaikan elemen UI dengan kebutuhan parameter Magnet Bumi.

2. Buat Controller Baru:

- Buat file `ui/controllers/magnet_controller.py`.
- Salin struktur dari controller lain, lalu sesuaikan logika validasi dan penyimpanan datanya.

3. Registrasi Halaman di Aplikasi (`app.py`):

- Impor kelas halaman baru:

```
from ui.views.halaman_magnet import HalamanMagnet
```
- Tambahkan konfigurasi halaman ke dalam variabel `PAGES` di kelas `App`:

```
PAGES = [  
    # ... halaman lain ...  
    {"class": HalamanMagnet, "name": "Magnet Bumi",  
     "icon": "🧲"},  
    # ...  
]
```
- Aplikasi secara otomatis akan merender tombol menu baru di Sidebar dan menangani navigasinya.

Skenario 3: Mengganti Kop Surat

Kasus: Perubahan identitas visual instansi.

1. **Siapkan File:** Pastikan file gambar baru berformat `.png` atau `.jpg`.
2. **Penempatan:** Timpa file lama di folder `assets/` dengan nama yang sama persis:
 - `kop_bmkg.jpg` (atau `.png`): Header untuk file PDF.
3. **Verifikasi:** Jalankan aplikasi. Kode `utils.get_resource_path()` akan otomatis memuat aset baru tersebut tanpa perlu perubahan kode program.

Skenario 4: Menambah Opsi Label pada Dropdown (Statis)

Kasus: Anda ingin mengubah atau menambah pilihan status pada form (misal: menambah status "Perlu Perbaikan" pada pengecekan kondisi).

Opsi-opsi seperti "Baik", "Rusak", atau "Tidak Ada" biasanya didefinisikan secara *hardcoded* di dalam file **View** karena sifatnya yang jarang berubah.

A. Untuk Laporan Seismograph:

1. Buka file `ui/views/halaman_seismograph.py`.
2. Cari metode `_build_common_checklist_form` atau bagian `form_kegiatan_maintenance`.
3. Temukan definisi variabel list opsi.

- *Kode Lama:*

```
options = ["Baik", "Tidak Baik", "Tidak Ada"]
```

- *Kode Baru:* Tambahkan string baru ke dalam list.

```
options = ["Baik", "Tidak Baik", "Tidak Ada", "Perlu Perbaikan"]
```

B. Untuk Laporan Preventif (Accelerometer/Intensity):

1. Buka file `ui/views/halaman_preventif.py`.
2. Cari definisi `ITEMS_CONFIG` di dalam metode `form_checklist_kondisi`.

Jika ingin menambah item baru yang diperiksa (misal: "Kondisi Pagar"), tambahkan *tuple* baru ke dalam list kategori yang sesuai:

```
("Umum", [
    ("shelter", "Shelter"),
    ("sekitar_peralatan", "Sekitar Peralatan"),
    ("pagar", "Kondisi Pagar") # <-- Item Baru
]),
```

3. Jangan lupa update Database (`database.py`) dan Controller (`preventif_controller.py`) untuk menyimpan kolom baru (`pagar`) tersebut (lihat *Skenario 1*).

C. Untuk Laporan Umum (WRS NG):

1. Buka file `ui/dialogs/tambah_kegiatan.py`.

2. Cari `get_pekerjaan()` yang diimpor dari `services/data_service.py`.
Untuk menambah opsi standar kegiatan, edit file `services/data_service.py`:

```
def get_pekerjaan():  
    return [  
        "Mengecek Kondisi Awal",  
        "Pekerjaan yang dilakukan",  
        "Pembersihan Virus", # <-- Opsi Baru  
        "Kesimpulan",  
        ...  
    ]
```

Skenario 5: Implementasi Input dengan Auto-complete

Kasus: Anda membuat input baru (misal: "Nama Teknisi Pendamping") dan ingin input tersebut bisa menyarankan nama secara otomatis dari database saat diketik.

Fitur ini menggunakan widget kustom `AutocompleteEntry` yang sudah tersedia di `ui/widgets/autocomplete_entry.py`.

Langkah 1: Persiapan Data di Controller Pastikan Controller memiliki data sumber (master data) yang akan ditampilkan sebagai saran.

- Buka Controller terkait (misal: `ui/controllers/seismograph_controller.py`).

Pastikan method `load_master_data` memuat data yang dibutuhkan.

```
# Contoh: Memuat data teknisi
self.master_teknisi_lain = db_service.get_all_petugas()
```

Langkah 2: Pasang Widget di View Ganti penggunaan `ttk.Entry` biasa dengan `AutocompleteEntry` pada file View (misal: `ui/views/halaman_seismograph.py`).

Import Widget:

```
from ui.widgets.autocomplete_entry import
AutocompleteEntry
```

Buat Widget: Ganti kode pembuatan entry lama dengan kode berikut:

```
# Membuat AutocompleteEntry
self.entry_teknisi_lain = AutocompleteEntry(
    frame,
    master_data=[], # Kosongkan dulu, nanti diisi
    Controller
    display_key='nama', # Kunci dictionary DB yang mau
    ditampilkan (misal: kolom 'nama')
    on_select_callback=None, # Fungsi yang dipanggil saat
    user klik saran (opsional)
    font=StyleConfig.FONT_NORMAL
)
self.entry_teknisi_lain.grid(row=..., column=...,
    sticky="ew")
```

Langkah 3: Sambungkan Data (Binding) Kembali ke Controller ([ui/controllers/seismograph_controller.py](#)), update method `load_data_into_ui` untuk menyuntikkan data ke widget tersebut.

```
def load_data_into_ui(self):
    # ... kode yang sudah ada ...

    # Tambahkan baris ini:
    # Mengisi widget dengan data dari self.master_teknisi_lain
    if hasattr(self.view, 'entry_teknisi_lain'):

self.view.entry_teknisi_lain.set_options(self.master_teknisi_la
in)
```

Hasil: Saat pengguna mengetik di kolom "Teknisi Pendamping", aplikasi akan memfilter daftar nama dari database dan menampilkannya dalam *dropdown*.